These guidelines may seem vague and theoretical, but they are critical pieces of thought when developing software, and by following these principles, we can integrate the SDK smoothly. Remember that even if the software is about building AI applications, it still needs to adhere to good practices for maintainability and reliability.

With that consideration, let's attempt to perform *incremental integration* of Vercel AI SDK into Astra AI next.

3.1.3 Practical integration: The Vercel AI SDK with Astra AI

Now that we've established our strategy, let's put it into practice. Our goal is to incrementally replace our existing API calls with the Vercel AI SDK's streamlined utilities. We'll start by addressing our first challenge: text generation.

> **NOTE** The source code for all the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch03.

Here's what we're going to build:

- A more efficient text generation system
- A seamless integration that preserves our existing features
- A good foundation for future improvements

Let's start with our first challenge: generating text using a prompt.

GENERATING TEXT USING THE GENERATETEXT UTILITY FUNCTION

The `generateText` function from the Vercel AI SDK core is designed to generate text in response to a given prompt. This function is particularly useful for noninteractive use cases, such as drafting emails, summarizing web pages, or any scenario where you need to produce written content based on a specific set of instructions. This function can also be used in place of the chat completions API for nonstreaming responses as well, since we only want to request the full response string. The general use of this function is shown in the following listing.

Listing 3.1 `generateText` example

```
import { generateText } from 'ai';

const { text } = await generateText({
  model,
  messages: [
      {
        role: 'system',
content: "I'm happy to assistyou in any way I can.
            How can I be of service today?",
      },
      { role: 'user', content: text },
    ],
});
```

generateText accepts an object parameter and returns another object with the result of the model API call.

model is the specific instance of the AI model that we use to generate text.